RejectionGym — BTS Cloud-ready version (matches your document requirements)

1) Project overview

Project title: RejectionGym – Challenge & Progress Platform
Short description: A cloud web app that helps users build confidence by completing daily challenges (social skills + personal growth). Users pick "challenge packs", complete tasks, track streaks/statistics, and generate a shareable progress report.

Project type: Cloud web application (3-tier)

2) Goal & value

- Helps users practice discomfort/rejection in a safe, structured way
- Gives measurable progress (streaks, difficulty progression, stats)
- Provides a **shareable** "progress report" page (good for demo + documentation aspect)

3) Scope (In / Out)

In scope (aligned with your doc)

User authentication (secure login/register, JWT, password hashing)
Creation of "projects" → here it's "profiles + goals" and "challenge plans"
Structured input → challenge creation form (difficulty, category, notes, target date)
File/image upload → optional but BTS-strong: upload proof (screenshot/photo/audio note)
Automatic generation of a readable page → "Progress Report" page (weekly/monthly) shareable by link
Cloud deployment → public URL, VM hosting, reverse proxy
D**ocker + CI/CD** → full pipeline

Out of scope (keep it realistic)

Real-time chat / social network (optional later)
AI coaching (optional later)
Mobile app (web responsive instead) (optional later)

4) Functional requirements (clear + teacher-friendly)

- Users can create an account and log in securely
- Users can browse preset challenge packs (social / career / fitness / comfort-zone)
- Users can create their own challenges and schedule them

- Users get a daily challenge plan (generated based on pack + difficulty)
- Users can mark challenges as done + add reflection notes
- Users can upload proof files (image/pdf/audio) linked to a challenge
- The system shows stats (streak, completion rate, difficulty progression)
- The system generates a **shareable progress report page** (public/private toggle)

5) Technology stack (BTS Cloud strong)

- **Frontend:** React
- **Backend:** Node.js + Express
- **Database:** PostgreSQL
- **File storage:** S3-compatible object storage (recommended: **MinIO** in Docker)
- **Containerization:** Docker + Docker Compose
- **CI/CD:** GitHub Actions (test → build → deploy)
- **Hosting:** Linux cloud VM (Nginx reverse proxy + HTTPS with Let's Encrypt)

6) Architecture & technical concept (3-tier + cloud features)

Three-tier architecture

1. **Frontend (React)**: UI, dashboard, challenge flow
2. **Backend API (Express)**: auth, business logic, report generation
3. **Database (PostgreSQL)**: users, challenges, completions, packs, files metadata

Extra BTS Cloud points (high-value)

- **Nginx reverse proxy** + HTTPS (Let's Encrypt)
- **Secrets via environment variables**
- **Health checks + logging**
- **Automated backups** (PostgreSQL dump nightly)
- **Role separation**: user vs admin (admin manages packs)

7) Project plan (high-level phases)

1. Planning + requirements + DB model
2. Backend API + Auth + core endpoints
3. Frontend UI (packs, daily plan, completion)
4. File upload + report generation page
5. Dockerization + CI/CD + deployment + tests + documentation + presentation

8) Risks & mitigation

- **Scope too big** → keep MVP strict (packs + daily plan + streak + report)
- **File upload security** → limit file types/size, store in MinIO, scan/validate
- **Deployment issues** → Docker Compose, reproducible pipeline, rollback strategy

9) Success criteria (what you'll prove in demo)

- App works online with a real domain/URL
- Users can register/login, complete challenges, see stats
- Report page is clear + shareable
- Deployment is stable and reproducible (Docker + CI/CD)
- Documentation explains architecture + setup clearly (BTS requirement)